

# **UNIVERSITY INSTITUTE OF COMPUTING**

## **PROJECT REPORT ON HOSPITAL MANAGEMENT SYSTEM**

Program Name: BCA

Subject Name/Code: Database Management  
System(23CAT-251)

**Submitted by:**

Name: Isha Rani

UID: 23BCA10129

Section: BCA – 4 “A”

**Submitted to:**

Name: Arvinder Singh

# **ABSTRACT**

## **Introduction:**

The Hospital Management System Database Project aims to simulate the core administrative functions of a hospital using a relational database model. In today's healthcare environment, the ability to manage, retrieve, and analyse patient and hospital-related data efficiently is crucial for ensuring smooth operations and better patient care. Traditional paper-based systems are prone to errors, time-consuming, and often lack the speed and flexibility needed in a fast-paced clinical setting.

This project addresses those limitations by presenting a structured and normalized SQL database system that organizes key data points such as patient details, doctor profiles, appointments, treatments, and billing records. By doing so, it improves data integrity, ensures accurate record-keeping, and facilitates efficient decision-making.

The system not only demonstrates practical database concepts but also reflects real-life scenarios that a hospital might face. For instance, it enables checking upcoming appointments, calculating total bills per patient, tracking completed treatments, and generating department-wise patient statistics.

This project is especially valuable for students of computer applications or information technology, as it provides hands-on exposure to designing relational schemas, implementing SQL queries, using joins, aggregate functions, and performing operations like SELECT, UPDATE, DELETE, and more. It encourages logical thinking, normalization techniques, and understanding of how data-driven solutions play a vital role in critical industries like healthcare.

# **Technique:**

The following techniques were used to design and implement the Hospital Management System database:

- Relational Database Design:

Tables such as Patient, Doctor, Department, Appointment, Treatment, and Bill are related using primary and foreign keys, maintaining referential integrity.

- SQL (Structured Query Language):

SQL commands such as CREATE, INSERT, SELECT, UPDATE, DELETE, and JOIN were used to build, populate, and manipulate the database.

- Normalization:

Data redundancy is minimized by organizing data into multiple related tables, following normalization principles.

- Data Integrity & Constraints:

Use of constraints like PRIMARY KEY, FOREIGN KEY, and data types ensures that only valid and consistent data is stored in the database.

- Data Retrieval using Queries:

Queries using JOIN, GROUP BY, WHERE, and aggregate functions like SUM() help extract meaningful reports from the system.

- Testing with Sample Data:

Sample data entries were inserted into each table to simulate realistic hospital operations and to test the queries effectively.

# **System Configuration:**

➤ Hardware Requirements:

- Processor: Intel i3 or above
- RAM: 4 GB (minimum)
- Storage: 500 MB free space

➤ Software Requirements:

- OS: Windows 10/11, Linux, or macOS
- DBMS: MySQL Server 8.0+ or XAMPP
- SQL Client: MySQL Workbench / phpMyAdmin / CLI
- Text Editor: VS Code / Notepad++

## **Input:**

The Hospital Management System Database receives its input primarily through Structured Query Language (SQL) commands executed in a MySQL environment. These inputs are in the form of SQL statements that define the database schema, insert relevant data, and manipulate or retrieve stored information.

Key types of input:

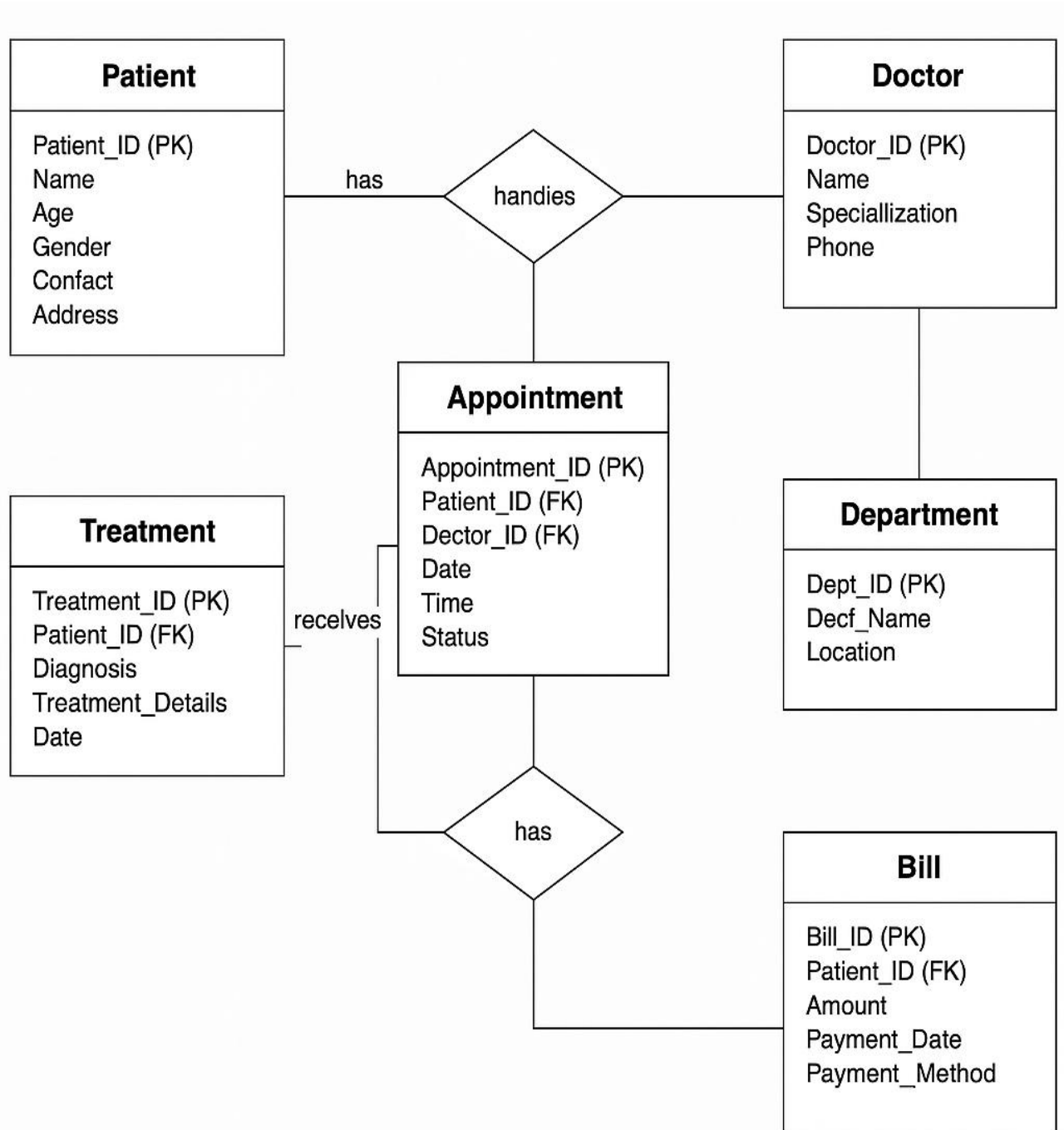
1. Data Definitions:

- The system begins by taking SQL CREATE TABLE statements to define the structure of various entities such as:
- Department
- Doctor
- Patient
- Appointment

- Treatment
- Bill
- 2. Data Entries (Insertion):
  - SQL INSERT commands are used to input initial sample records for:
    - Department details (like name and location)
    - Doctor profiles (name, specialization, contact, department ID)
    - Patient information (name, age, gender, contact, address)
    - Appointment scheduling (linked with doctor and patient)
    - Treatment history (diagnosis and treatment description)
    - Billing records (amount, payment mode and date)
- 3. Data Manipulation and Retrieval:
  - Input also includes SELECT, UPDATE, DELETE, and JOIN queries that:
    - Retrieve appointment details
    - Update patient contact information
    - Delete canceled appointments
    - Summarize total billing per patient
    - Show completed appointments, etc.

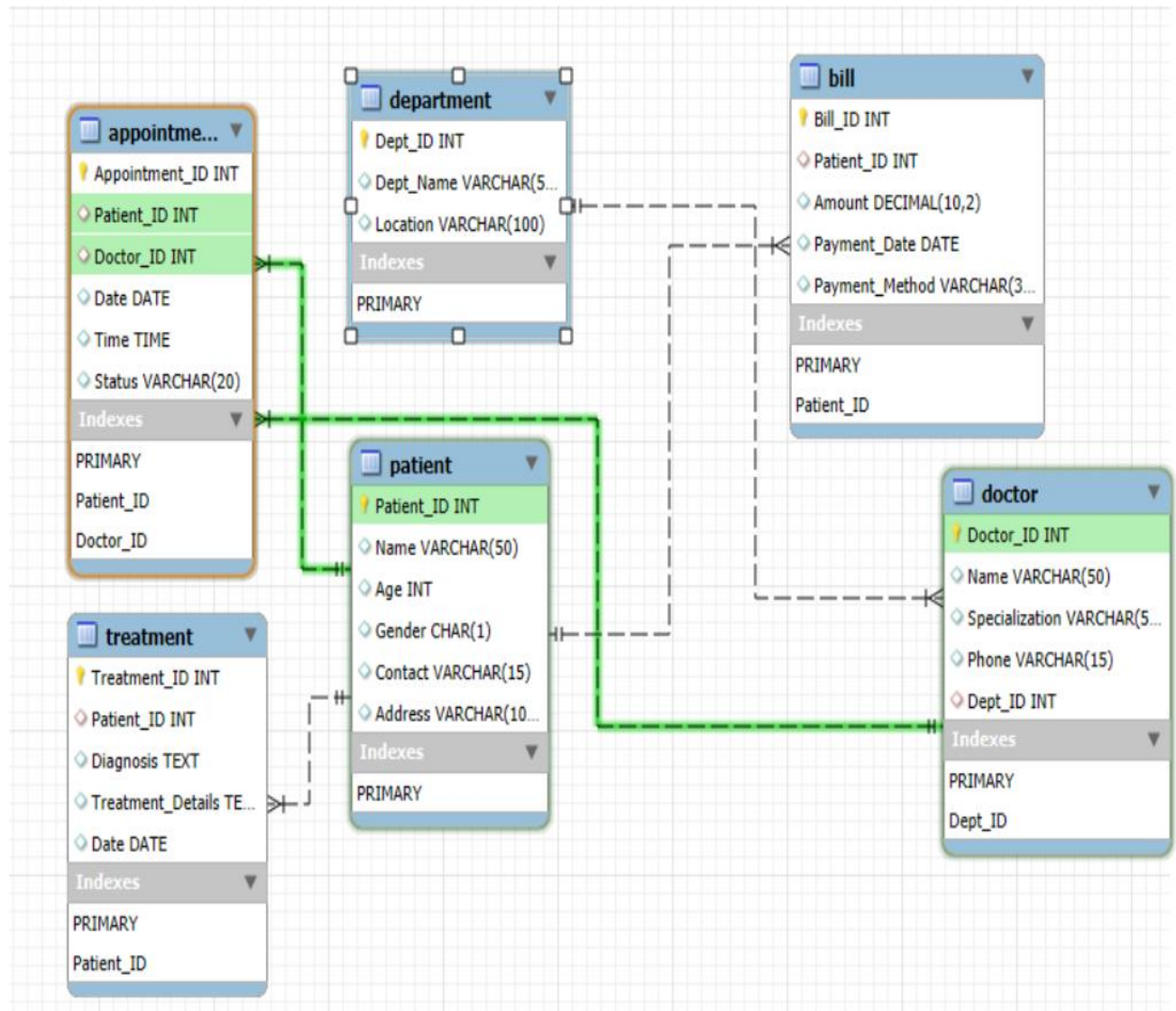
These inputs are executed via SQL scripts in MySQL Workbench.

## ER DIAGRAM:

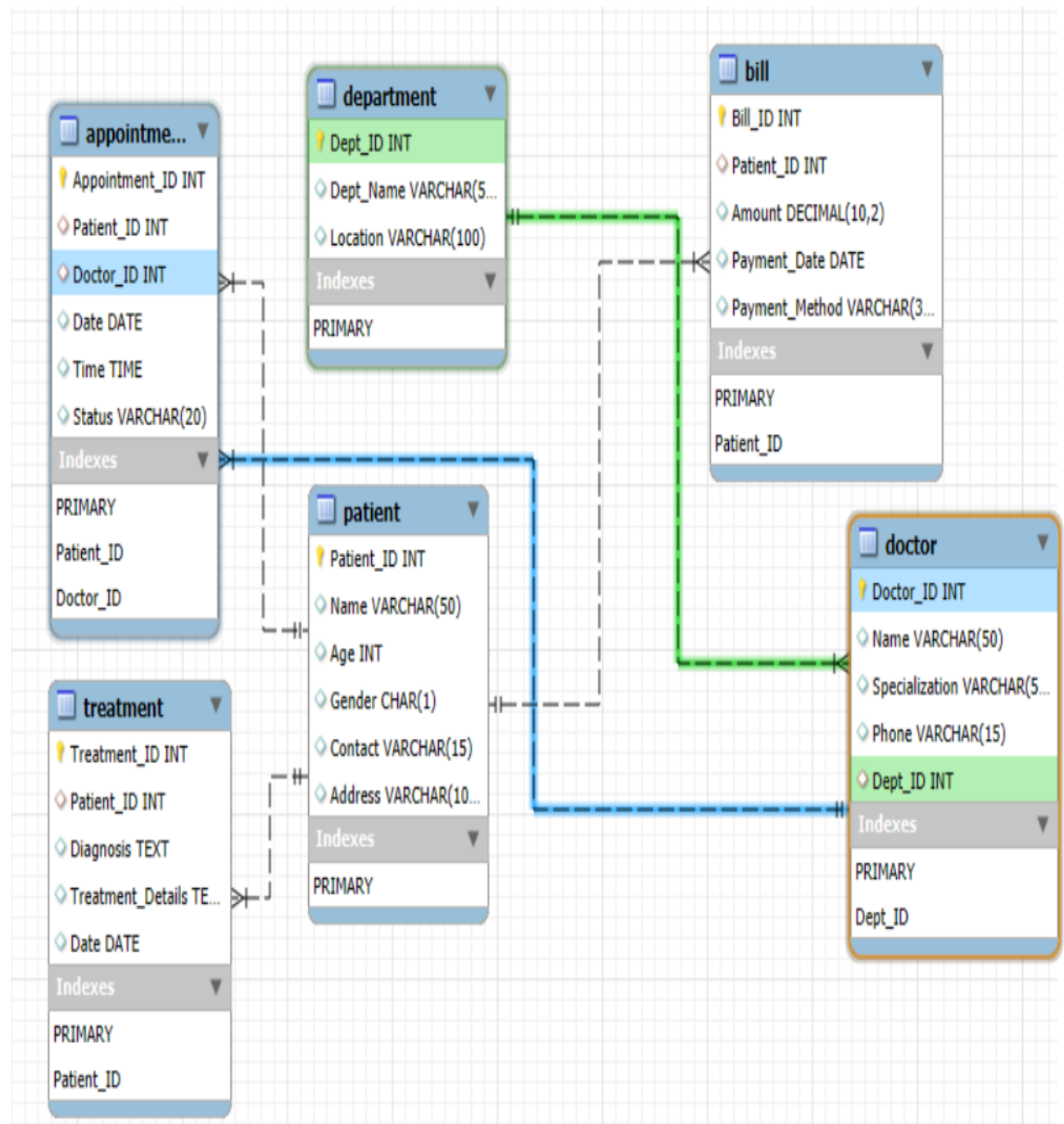


## TABLE REALTION:

### 1. Table appointment

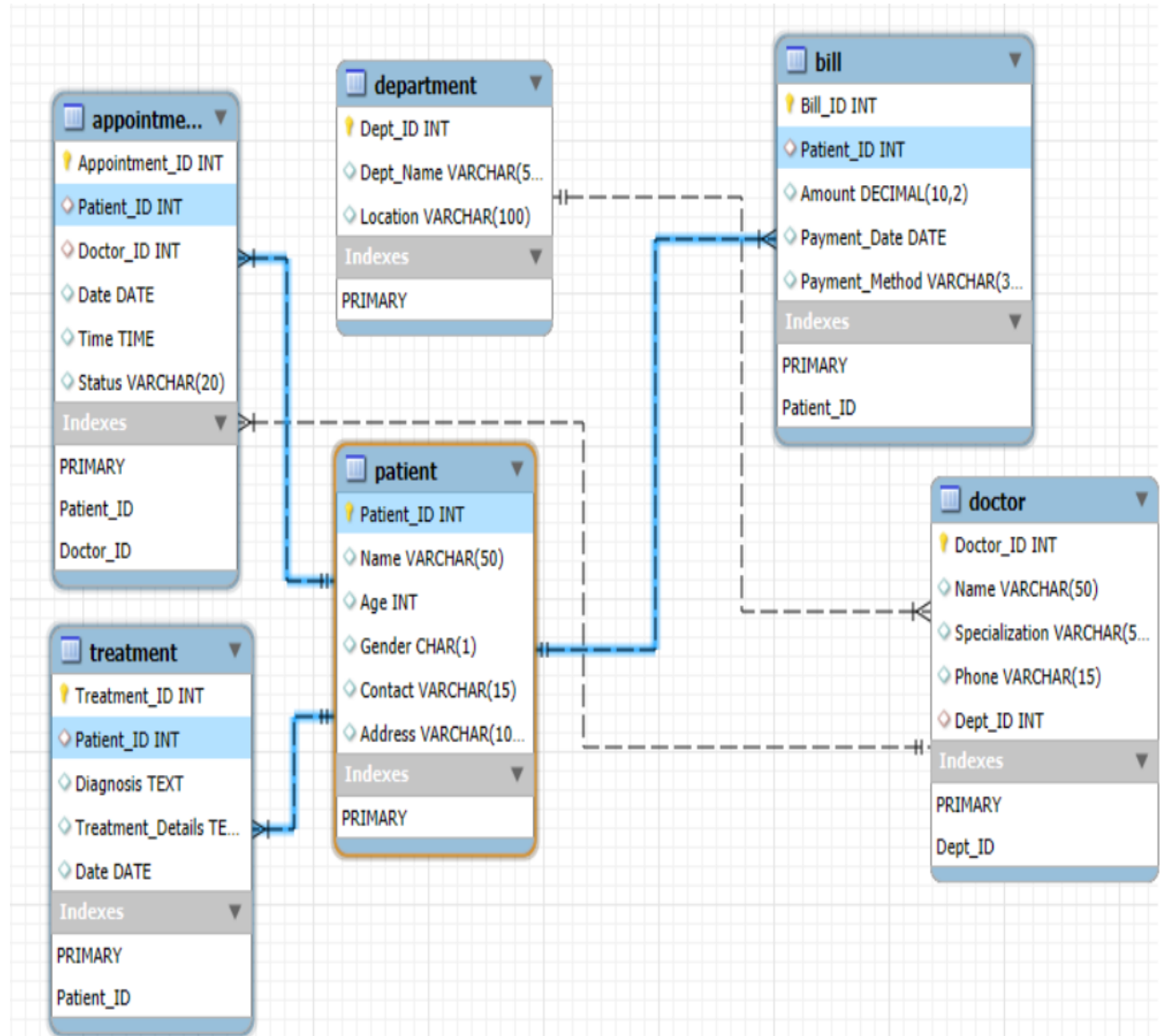


## Table doctor:

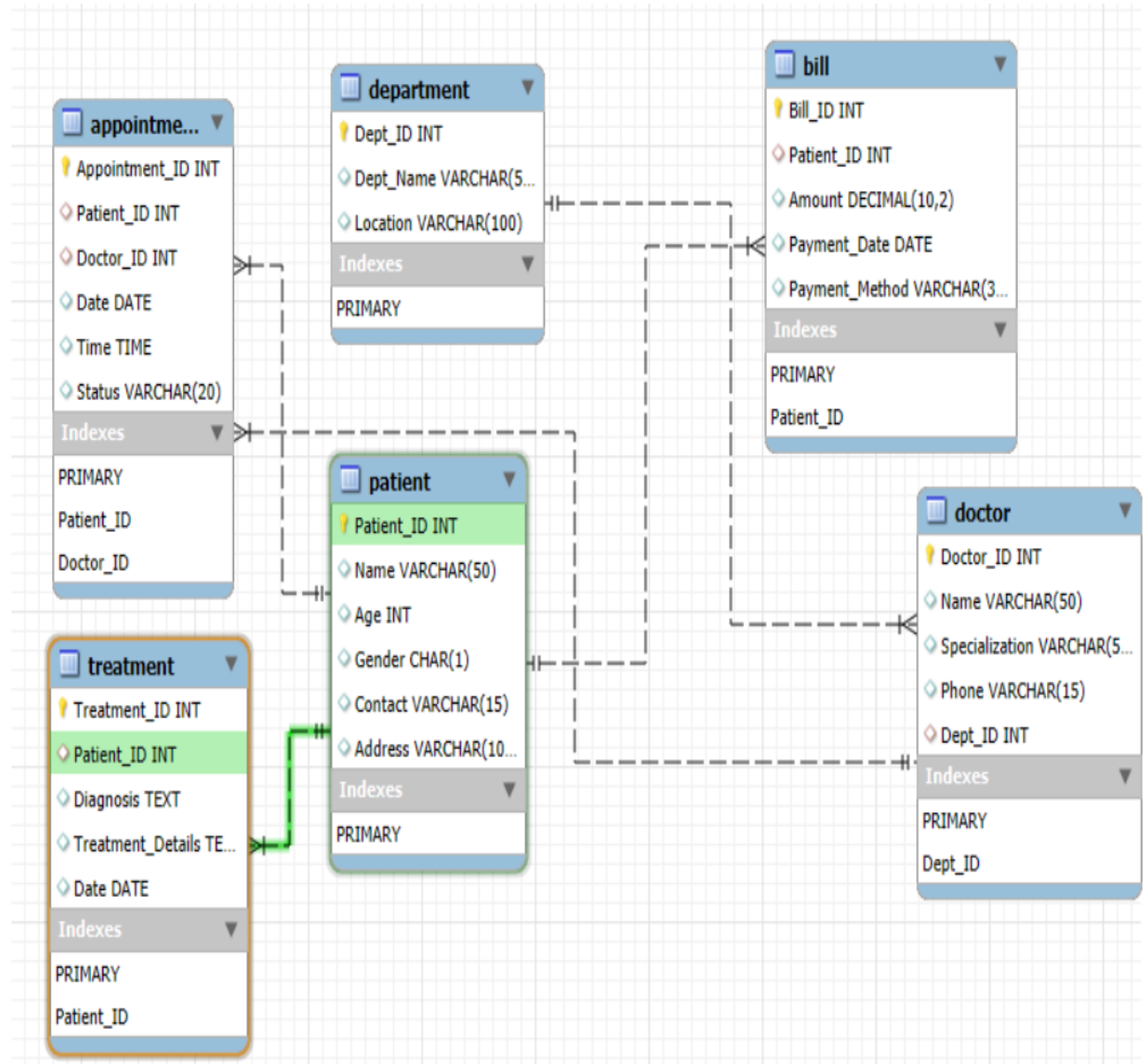




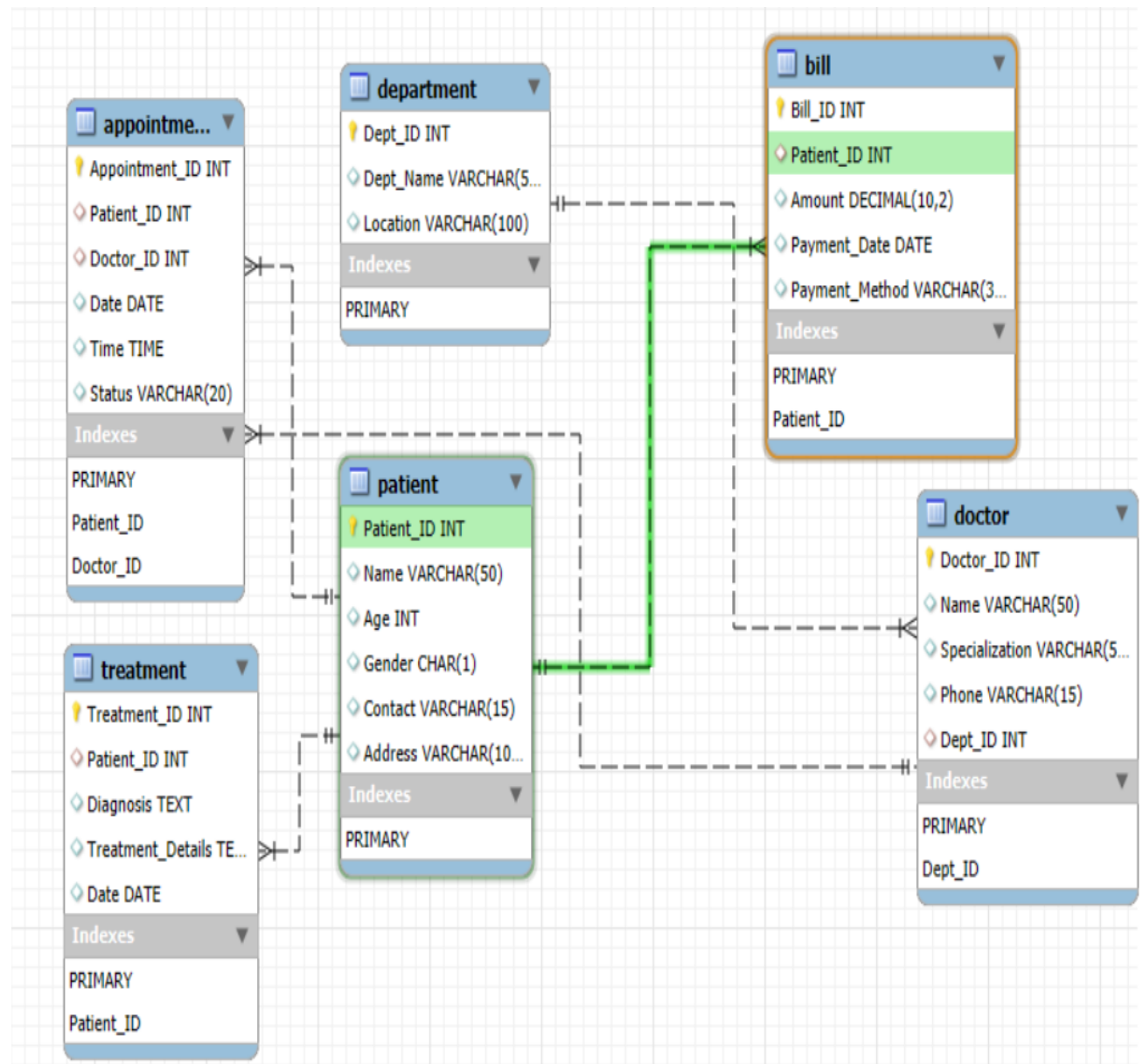
## Table patient:



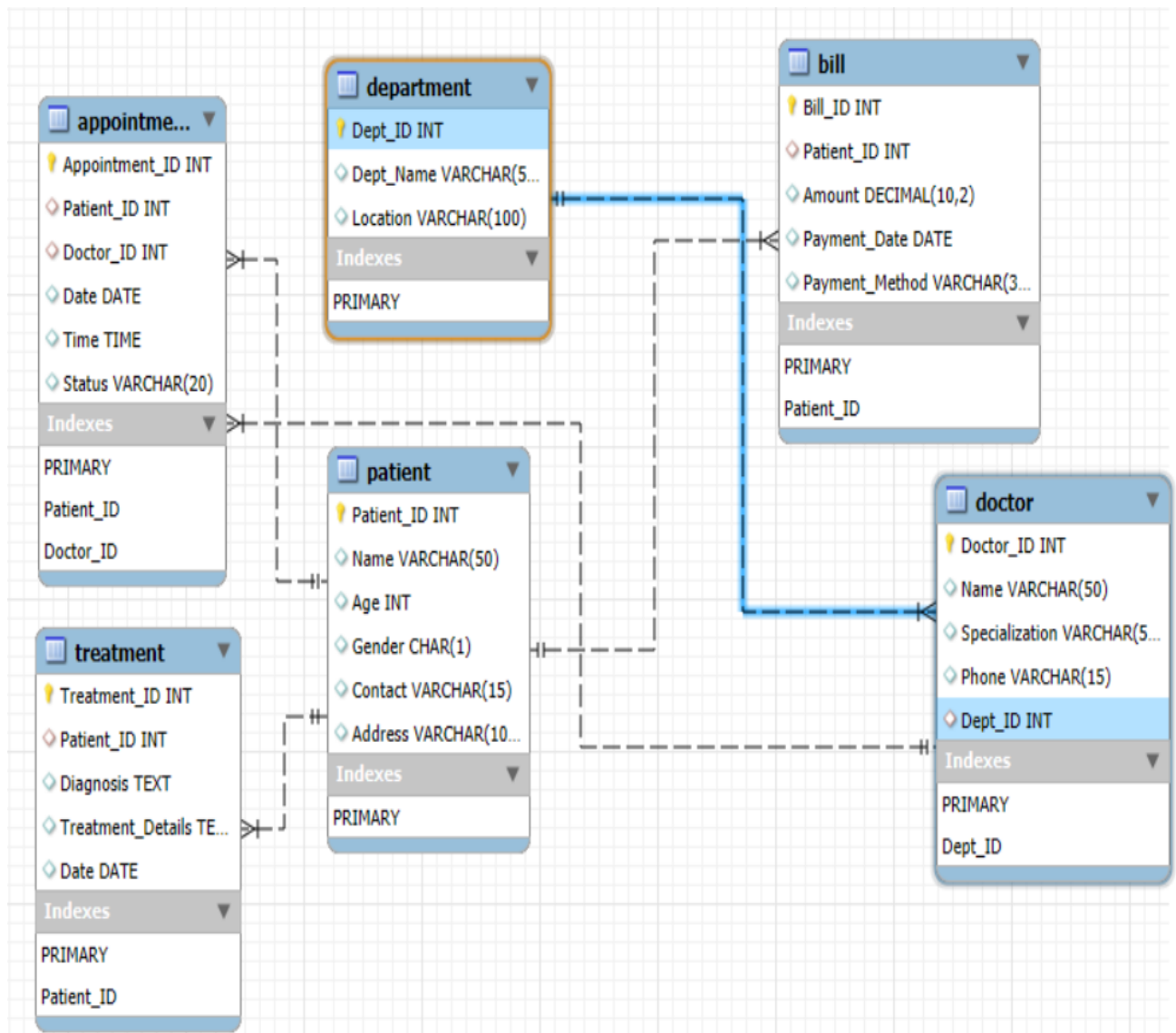
## Table treatment:



## Table bill:



## Table department:



# TABULAR FORMAT:

## 1. Department

| <u>Attribute</u> | <u>Data Type</u>    | <u>Key</u>         | <u>Description</u>              |
|------------------|---------------------|--------------------|---------------------------------|
| <u>Dept ID</u>   | <u>INT</u>          | <u>Primary Key</u> | <u>Unique ID for department</u> |
| <u>Dept Name</u> | <u>VARCHAR(50)</u>  |                    | <u>Name of the department</u>   |
| <u>Location</u>  | <u>VARCHAR(100)</u> |                    | <u>Location of department</u>   |

## 2. Doctor

| <u>Attribute</u>      | <u>Data Type</u>   | <u>Key</u>         | <u>Description</u>                    |
|-----------------------|--------------------|--------------------|---------------------------------------|
| <u>Doctor ID</u>      | <u>INT</u>         | <u>Primary Key</u> | <u>Unique ID for doctor</u>           |
| <u>Name</u>           | <u>VARCHAR(50)</u> |                    | <u>Doctor's name</u>                  |
| <u>Specialization</u> | <u>VARCHAR(50)</u> |                    | <u>Area of specialization</u>         |
| <u>Phone</u>          | <u>VARCHAR(15)</u> |                    | <u>Contact number</u>                 |
| <u>Dept ID</u>        | <u>INT</u>         | <u>Foreign Key</u> | <u>References Department(Dept ID)</u> |

### 3. Patient

| <u>Attribute</u>  | <u>Data Type</u>    | <u>Key</u>         | <u>Description</u>            |
|-------------------|---------------------|--------------------|-------------------------------|
| <u>Patient ID</u> | <u>INT</u>          | <u>Primary Key</u> | <u>Unique ID for patient</u>  |
| <u>Name</u>       | <u>VARCHAR(50)</u>  |                    | <u>Patient's name</u>         |
| <u>Age</u>        | <u>INT</u>          |                    | <u>Patient's age</u>          |
| <u>Gender</u>     | <u>CHAR(1)</u>      |                    | <u>'M' or 'F'</u>             |
| <u>Contact</u>    | <u>VARCHAR(15)</u>  |                    | <u>Contact number</u>         |
| <u>Address</u>    | <u>VARCHAR(100)</u> |                    | <u>Address of the patient</u> |

### 4. Appointment

| <u>Attribute</u>      | <u>Data Type</u>   | <u>Key</u>         | <u>Description</u>                     |
|-----------------------|--------------------|--------------------|----------------------------------------|
| <u>Appointment ID</u> | <u>INT</u>         | <u>Primary Key</u> | <u>Unique appointment ID</u>           |
| <u>Patient ID</u>     | <u>INT</u>         | <u>Foreign Key</u> | <u>References Patient(Patient ID)</u>  |
| <u>Doctor ID</u>      | <u>INT</u>         | <u>Foreign Key</u> | <u>References Doctor(Doctor ID)</u>    |
| <u>Date</u>           | <u>DATE</u>        |                    | <u>Date of appointment</u>             |
| <u>Time</u>           | <u>TIME</u>        |                    | <u>Time of appointment</u>             |
| <u>Status</u>         | <u>VARCHAR(20)</u> |                    | <u>Scheduled, Completed, Cancelled</u> |

## 5. Treatment

| <u>Attribute</u>         | <u>Data Type</u> | <u>Key</u>         | <u>Description</u>                    |
|--------------------------|------------------|--------------------|---------------------------------------|
| <u>Treatment ID</u>      | <u>INT</u>       | <u>Primary Key</u> | <u>Unique treatment ID</u>            |
| <u>Patient ID</u>        | <u>INT</u>       | <u>Foreign Key</u> | <u>References Patient(Patient ID)</u> |
| <u>Diagnosis</u>         | <u>TEXT</u>      |                    | <u>Diagnosis description</u>          |
| <u>Treatment Details</u> | <u>TEXT</u>      |                    | <u>Treatment provided</u>             |
| <u>Date</u>              | <u>DATE</u>      |                    | <u>Date of treatment</u>              |

## 6. Bill

| <u>Attribute</u>      | <u>Data Type</u>     | <u>Key</u>         | <u>Description</u>                    |
|-----------------------|----------------------|--------------------|---------------------------------------|
| <u>Bill ID</u>        | <u>INT</u>           | <u>Primary Key</u> | <u>Unique bill ID</u>                 |
| <u>Patient ID</u>     | <u>INT</u>           | <u>Foreign Key</u> | <u>References Patient(Patient ID)</u> |
| <u>Amount</u>         | <u>DECIMAL(10,2)</u> |                    | <u>Total amount charged</u>           |
| <u>Payment Date</u>   | <u>DATE</u>          |                    | <u>Date of payment</u>                |
| <u>Payment Method</u> | <u>VARCHAR(30)</u>   |                    | <u>e.g., Cash, Card, Online, Insu</u> |

## IMPLEMENTATION:

### Tables Created:

|   |   |          |                                                                                                                 |                   |
|---|---|----------|-----------------------------------------------------------------------------------------------------------------|-------------------|
| ✓ | 3 | 22:49:41 | CREATE TABLE Department ( Dept_ID INT PRIMARY KEY, Dept_Name VARCHAR(50), Location VARCHAR(100))                | 0 row(s) affected |
| ✓ | 4 | 22:49:44 | CREATE TABLE Doctor ( Doctor_ID INT PRIMARY KEY, Name VARCHAR(50), Specialization VARCHAR(50), Phone VAR...     | 0 row(s) affected |
| ✓ | 5 | 22:49:50 | CREATE TABLE Patient ( Patient_ID INT PRIMARY KEY, Name VARCHAR(50), Age INT, Gender CHAR(1), Contact VA...     | 0 row(s) affected |
| ✓ | 6 | 22:49:58 | CREATE TABLE Appointment ( Appointment_ID INT PRIMARY KEY, Patient_ID INT, Doctor_ID INT, Date DATE, Time ...   | 0 row(s) affected |
| ✓ | 7 | 22:50:05 | CREATE TABLE Treatment ( Treatment_ID INT PRIMARY KEY, Patient_ID INT, Diagnosis TEXT, Treatment_Details TEX... | 0 row(s) affected |
| ✓ | 8 | 22:50:12 | CREATE TABLE Bill ( Bill_ID INT PRIMARY KEY, Patient_ID INT, Amount DECIMAL(10,2), Payment_Date DATE, Paym...   | 0 row(s) affected |

## SQL Queries Performed with Output:

### 1. Display all patients

```
SELECT * FROM Patient;
```

- **Purpose:**

Displays a complete list of all patients registered in the hospital system, including personal details like name, age, gender, address, and contact information.

| Patient_ID | Name    | Age  | Gender | Contact    | Address |
|------------|---------|------|--------|------------|---------|
| 201        | Alice   | 30   | F      | 9998887771 | Area 1  |
| 202        | Bob     | 45   | M      | 9998887772 | Area 2  |
| 203        | Charlie | 22   | M      | 9998887773 | Area 3  |
| 204        | Daisy   | 28   | F      | 9998887774 | Area 4  |
| 205        | Ethan   | 35   | M      | 9998887775 | Area 5  |
| 206        | Fiona   | 50   | F      | 9998887776 | Area 6  |
| 207        | George  | 60   | M      | 9998887777 | Area 7  |
| 208        | Hannah  | 40   | F      | 9998887778 | Area 8  |
| 209        | Ian     | 32   | M      | 9998887779 | Area 9  |
| 210        | Jane    | 55   | F      | 9998887780 | Area 10 |
| NULL       | NULL    | NULL | NULL   | NULL       | NULL    |



## 2. List appointments with doctor and patient names

```
SELECT A.Appointment_ID, P.Name AS Patient, D.Name AS Doctor, A.Date, A.Time, A.Status
```

```
FROM Appointment A
```

```
JOIN Patient P ON A.Patient_ID = P.Patient_ID
```

```
JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID;
```

- **Purpose:**  
Provides a detailed view of each appointment, showing the involved patient and doctor names along with the appointment's date, time, and status.
- **Use Case:**  
Helps hospital staff quickly check scheduled meetings and track patient–doctor interactions.

| Appointment_ID | Patient | Doctor     | Date       | Time     | Status    |
|----------------|---------|------------|------------|----------|-----------|
| 301            | Alice   | Dr. Sharma | 2025-04-01 | 10:00:00 | Completed |
| 302            | Bob     | Dr. Mehta  | 2025-04-02 | 11:00:00 | Scheduled |
| 303            | Charlie | Dr. Roy    | 2025-04-03 | 09:30:00 | Completed |
| 304            | Daisy   | Dr. Verma  | 2025-04-04 | 14:00:00 | Cancelled |
| 305            | Ethan   | Dr. Iyer   | 2025-04-05 | 15:30:00 | Completed |
| 306            | Fiona   | Dr. Das    | 2025-04-06 | 12:00:00 | Scheduled |
| 307            | George  | Dr. Gupta  | 2025-04-07 | 13:00:00 | Completed |
| 308            | Hannah  | Dr. Reddy  | 2025-04-08 | 16:00:00 | Completed |
| 309            | Ian     | Dr. Joshi  | 2025-04-09 | 10:30:00 | Scheduled |
| 310            | Jane    | Dr. Khan   | 2025-04-10 | 11:30:00 | Completed |

### 3. Count total number of patients per department

```
SELECT Dept_Name, COUNT(A.Patient_ID) AS Total_Patients  
FROM Appointment A  
JOIN Doctor D ON A.Doctor_ID = D.Doctor_ID  
JOIN Department DP ON D.Dept_ID = DP.Dept_ID  
GROUP BY Dept_Name;
```

- **Purpose:**  
Calculates how many patients are associated with each department by counting appointments.
- **Use Case:**  
Useful for department-wise workload analysis and resource planning.

| Dept_Name   | Total_Patients |
|-------------|----------------|
| Cardiology  | 1              |
| Neurology   | 1              |
| Orthopedics | 1              |
| Pediatrics  | 1              |
| Dermatology | 1              |
| ENT         | 1              |
| Oncology    | 1              |
| Gynecology  | 1              |
| Psychiatry  | 1              |
| Urology     | 1              |

#### 4. List treatments of a specific patient

```
SELECT * FROM Treatment WHERE Patient_ID = 201;
```

- **Purpose:**  
Shows all treatments received by a specific patient, including diagnosis, procedures, and prescribed medicines.
- **Use Case:**  
Critical for medical staff to track ongoing or past treatments of a patient.

| Treatment_ID | Patient_ID | Diagnosis    | Treatment_Details     | Date       |
|--------------|------------|--------------|-----------------------|------------|
| 401          | 201        | Hypertension | Prescribed medication | 2025-04-01 |
| NULL         | NULL       | NULL         | NULL                  | NULL       |

#### 5. Update patient contact number

```
UPDATE Patient SET Contact = '8887776666' WHERE Patient_ID = 202;
```

- **Purpose:**  
Modifies the contact number of a patient in case of changes or corrections.
- **Use Case:**  
Keeps patient contact information accurate for communication and emergencies.

#### 6. Delete an appointment

```
DELETE FROM Appointment WHERE Appointment_ID = 304;
```

- **Purpose:**  
Removes an appointment record from the database, typically used when appointments are cancelled.
- **Use Case:**  
Prevents clutter and maintains data relevancy in the system.

## 7. Show total bill paid by each patient

```
SELECT Patient_ID, SUM(Amount) AS Total_Amount  
FROM Bill  
GROUP BY Patient_ID;
```

- **Purpose:**  
Sums up the total billing amount per patient.
- **Use Case:**  
Helps in generating patient invoices and financial summaries.

| Patient_ID | Total_Amount |
|------------|--------------|
| 201        | 1500.00      |
| 202        | 2000.00      |
| 203        | 10000.00     |
| 204        | 500.00       |
| 205        | 1200.00      |
| 206        | 800.00       |
| 207        | 30000.00     |
| 208        | 1000.00      |
| 209        | 1800.00      |
| 210        | 15000.00     |

## 8. List doctors and their department names

```
SELECT D.Name, D.Specialization, DP.Dept_Name
```

```
FROM Doctor D
```

```
JOIN Department DP ON D.Dept_ID = DP.Dept_ID;
```

- **Purpose:**  
Combines doctor and department information for a complete overview.
- **Use Case:**  
Enables management to view doctors' qualifications and department affiliations.

| Name       | Specialization | Dept_Name   |
|------------|----------------|-------------|
| Dr. Sharma | Cardiologist   | Cardiology  |
| Dr. Mehta  | Neurologist    | Neurology   |
| Dr. Roy    | Orthopedic     | Orthopedics |
| Dr. Verma  | Pediatrician   | Pediatrics  |
| Dr. Iyer   | Dermatologist  | Dermatology |
| Dr. Das    | ENT Specialist | ENT         |
| Dr. Gupta  | Oncologist     | Oncology    |
| Dr. Reddy  | Gynecologist   | Gynecology  |
| Dr. Joshi  | Psychiatrist   | Psychiatry  |
| Dr. Khan   | Urologist      | Urology     |

## 9. Display all completed appointments

```
SELECT * FROM Appointment WHERE Status = 'Completed';
```

- **Purpose:**  
Retrieves all appointments that have already taken place.
- **Use Case:**  
Used for reviewing daily operations or performance reporting.

| Appointment_ID | Patient_ID | Doctor_ID | Date       | Time     | Status    |
|----------------|------------|-----------|------------|----------|-----------|
| 301            | 201        | 101       | 2025-04-01 | 10:00:00 | Completed |
| 303            | 203        | 103       | 2025-04-03 | 09:30:00 | Completed |
| 305            | 205        | 105       | 2025-04-05 | 15:30:00 | Completed |
| 307            | 207        | 107       | 2025-04-07 | 13:00:00 | Completed |
| 308            | 208        | 108       | 2025-04-08 | 16:00:00 | Completed |
| 310            | 210        | 110       | 2025-04-10 | 11:30:00 | Completed |
| NULL           | NULL       | NULL      | NULL       | NULL     | NULL      |

## 10. Show patients who paid more than 5000

```
SELECT * FROM Bill WHERE Amount > 5000;
```

- **Purpose:**  
Identifies patients with high medical expenses.
- **Use Case:**  
Useful for finance teams or for identifying cases for insurance follow-up.

| Bill_ID | Patient_ID | Amount   | Payment_Date | Payment_Method |
|---------|------------|----------|--------------|----------------|
| 503     | 203        | 10000.00 | 2025-04-03   | Online         |
| 507     | 207        | 30000.00 | 2025-04-07   | Insurance      |
| 510     | 210        | 15000.00 | 2025-04-10   | Online         |
| NULL    | NULL       | NULL     | NULL         | NULL           |

# **SUMMARY:**

The **Hospital Management System Database Project** is a comprehensive relational database designed to simulate and manage core functionalities of a hospital digitally. It emphasizes efficient data organization, accessibility, and consistency using **Structured Query Language (SQL)**. The system comprises multiple interrelated tables, including Department, Doctor, Patient, Appointment, Treatment, and Bill, each representing real-world entities and their interactions.

The primary objective is to streamline hospital workflows by digitizing patient records, appointment scheduling, doctor assignments, treatment tracking, and billing management. By doing so, the project showcases how relational databases can effectively reduce human error, improve information retrieval, and maintain data accuracy.

The use of **foreign keys** and **referential integrity** ensures that data across all tables remains consistent and connected. SQL queries demonstrate the practical application of database concepts, such as:

- **Joins** to combine data from multiple tables for reports (e.g., patient appointments with doctors)
- **Aggregate functions** to calculate totals and summaries (e.g., total billing amounts)
- **Conditional filters** to fetch specific records (e.g., completed appointments or high-value bills)

This project not only simulates real-world hospital operations but also helps students gain hands-on experience with database design, normalization, and query optimization. It proves to be a valuable academic exercise in understanding how databases support large-scale, mission-critical systems like those found in healthcare.

## **CONCLUSION:**

The **Hospital Management System database project** serves as a robust and practical demonstration of how database technologies—particularly SQL—can be effectively employed to model and manage real-world operations within a healthcare environment. It successfully replicates the critical components of hospital administration, such as maintaining records for departments, doctors, patients, appointments, treatments, and financial transactions, through the use of structured relational tables.

Throughout the project, we employed a variety of **SQL operations**, including data insertion, updating, deletion, joins, and aggregation, to reflect real-time processes and interactions. These operations allowed for **efficient data retrieval**, enabled **integrity between related tables**, and ensured **consistency in patient care records**. By implementing this system, the challenges of manual record-keeping—such as redundancy, data loss, and errors—were mitigated, making the entire hospital workflow more seamless and reliable.

Moreover, this project highlights the importance of database normalization and the enforcement of **primary and foreign key constraints**, which are essential for building scalable and maintainable systems. It also illustrates how decision-makers in healthcare can leverage SQL queries for generating analytical reports and drawing actionable insights from stored data.

Overall, this initiative not only enhanced our understanding of **relational database design and SQL programming**, but also emphasized the value of data-driven solutions in complex environments like healthcare. It stands as a testament to how well-architected database systems can significantly improve **operational efficiency, data security, and service delivery** in critical sectors.





**CHANDIGARH  
UNIVERSITY**

Discover. Learn. Empower.

**THANK  
YOU!**